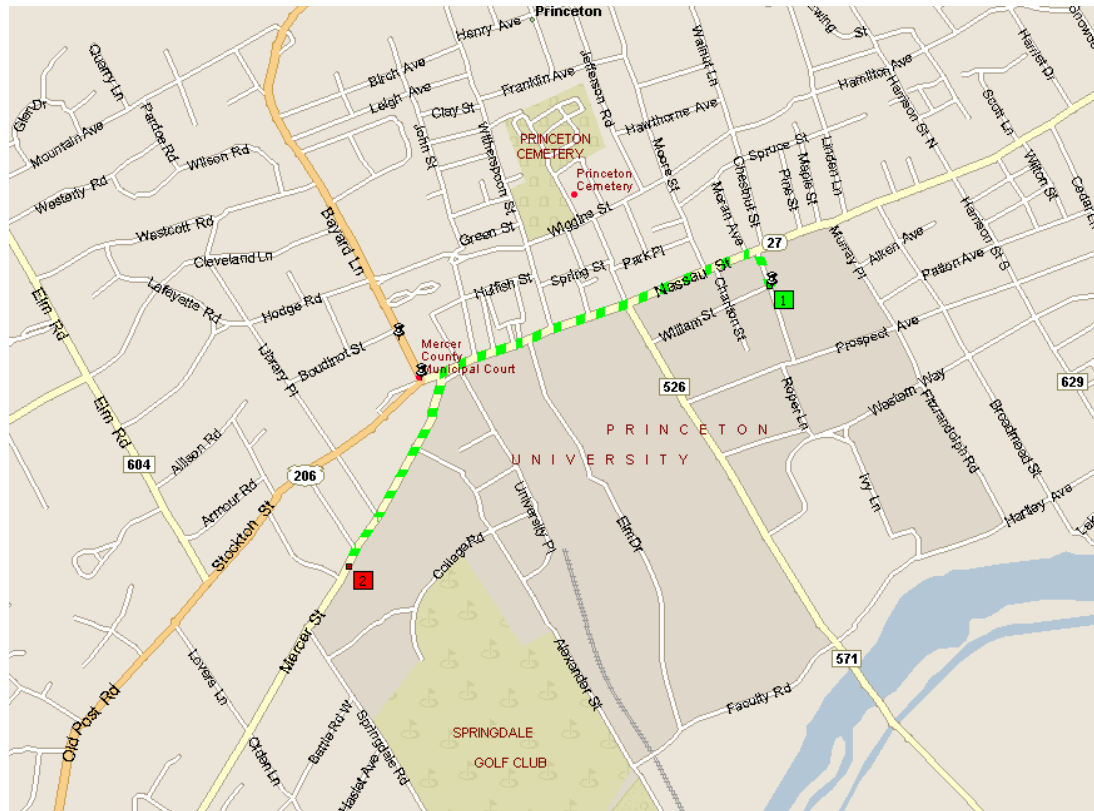


# Shortest Paths in a Graph



shortest path from Princeton CS department to Einstein's house

## Car navigation



# Shortest path applications

Shortest Path Problem

- PERT/CPM.
- Map routing.
- Seam carving.
- Robot navigation.
- Texture mapping.
- Typesetting in LaTeX.
- Urban traffic planning.
- Telemarketer operator scheduling.
- Routing of telecommunications messages.
- Network routing protocols (OSPF, BGP, RIP).
- Optimal truck routing through given traffic congestion pattern.

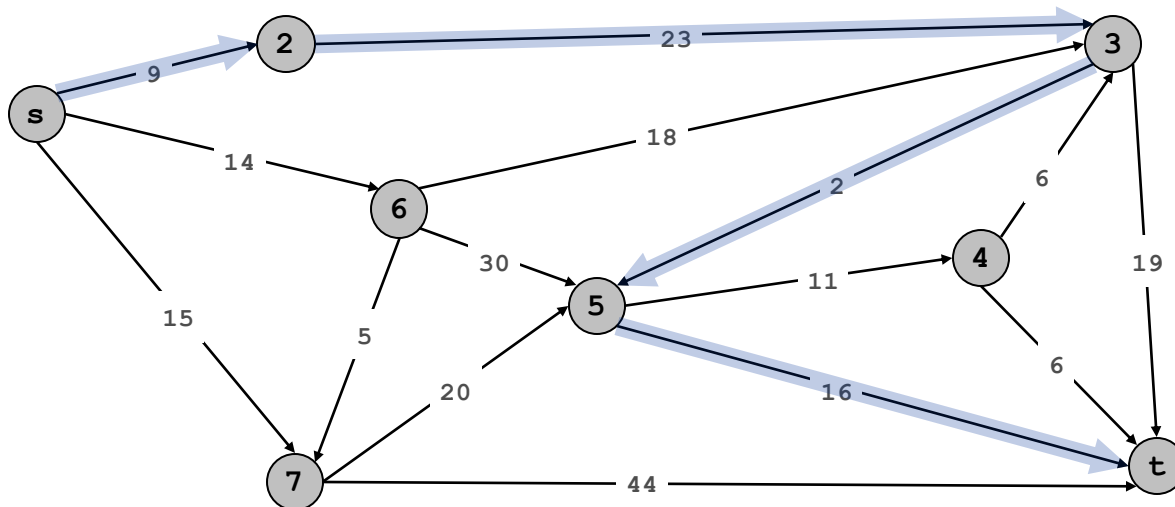
# Shortest Path Problem

## Shortest path network.

- Directed graph  $G = (V, E)$  with positive edges.
- Source  $s$ , destination  $t$ .
- Length  $\ell_e$  = length of edge  $e$ .

Shortest path problem: find **shortest directed path** from  $s$  to  $t$ .

cost of path = sum of edge costs in path



Cost of path  $s-2-3-5-t$   
 $= 9 + 23 + 2 + 16$   
 $= 50.$

# Dijkstra's Algorithm

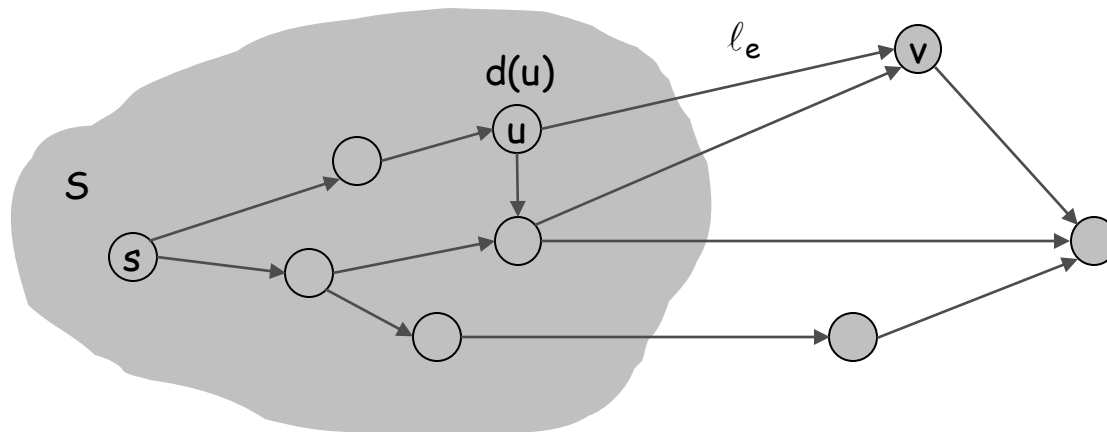
## Dijkstra's algorithm.

- Maintain a set of **explored nodes**  $S$  for which we have determined the shortest path distance  $d(u)$  from  $s$  to  $u$ .
- Initialize  $S = \{s\}$ ,  $d(s) = 0$ .
- Repeatedly choose unexplored node  $v$  (i.e.,  $v \notin S$ ) that minimizes

$$\pi(v) = \min_{e = (u,v) : u \in S} d(u) + \ell_e,$$

add  $v$  to  $S$ , and set  $d(v) = \pi(v)$ .

shortest path to some  $u$  in explored part, followed by a single edge  $(u, v)$



# Dijkstra's Algorithm

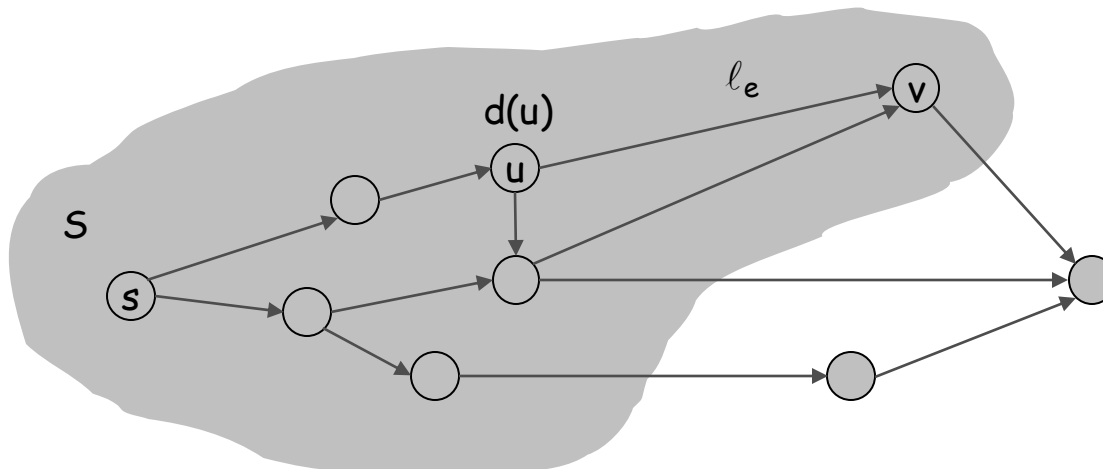
## Dijkstra's algorithm.

- Maintain a set of **explored nodes**  $S$  for which we have determined the shortest path distance  $d(u)$  from  $s$  to  $u$ .
- Initialize  $S = \{s\}$ ,  $d(s) = 0$ .
- Repeatedly choose unexplored node  $v$  which minimizes

$$\pi(v) = \min_{e = (u, v) : u \in S} d(u) + \ell_e,$$

add  $v$  to  $S$ , and set  $d(v) = \pi(v)$ .

shortest path to some  $u$  in explored part, followed by a single edge  $(u, v)$



DEMO  
(05demo-dijkstra)



# Dijkstra's algorithm

$d[s] \leftarrow 0$

for each  $v \in V - \{s\}$

do  $d[v] \leftarrow \infty$

$S \leftarrow \emptyset$

$Q \leftarrow V$

while  $Q \neq \emptyset$

( $Q$  min-priority queue for  $V - S$ )

do  $u \leftarrow \text{DELETE-MIN}(Q)$

$S \leftarrow S \cup \{u\}$

for each  $v \in \text{Adj}[u]$

do if  $d[v] > d[u] + w(u, v)$

then  $d[v] \leftarrow d[u] + w(u, v)$

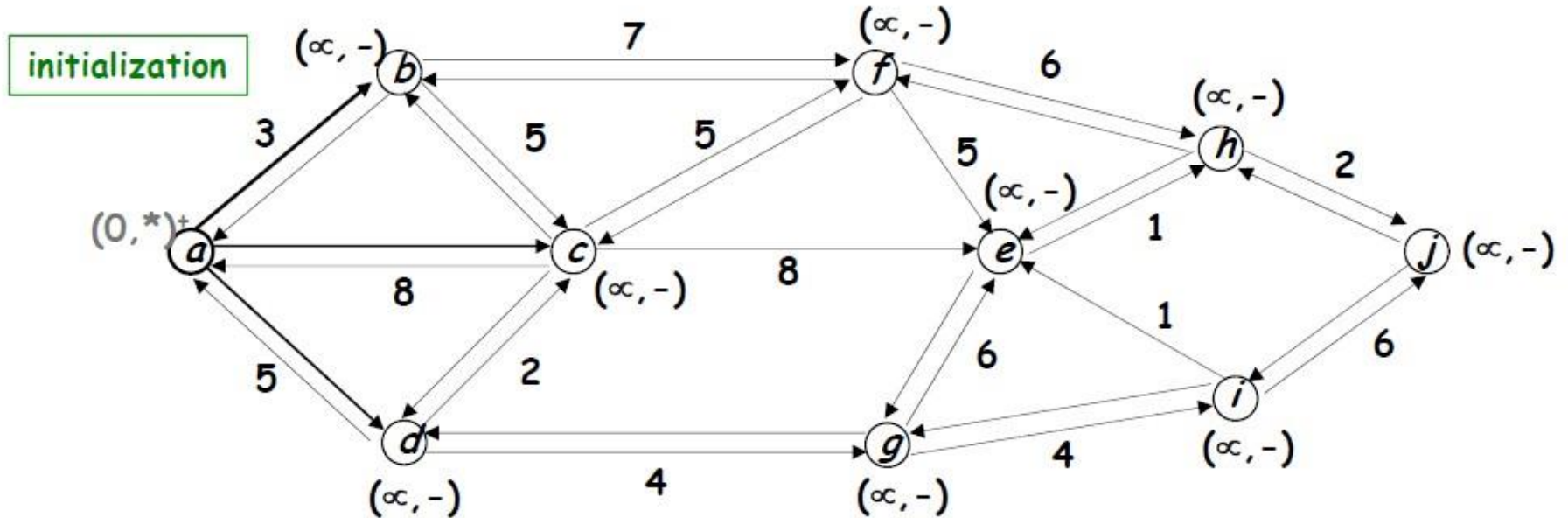
(Implicit **DECREASE-KEY**)

*initialization*

*relaxation steps*

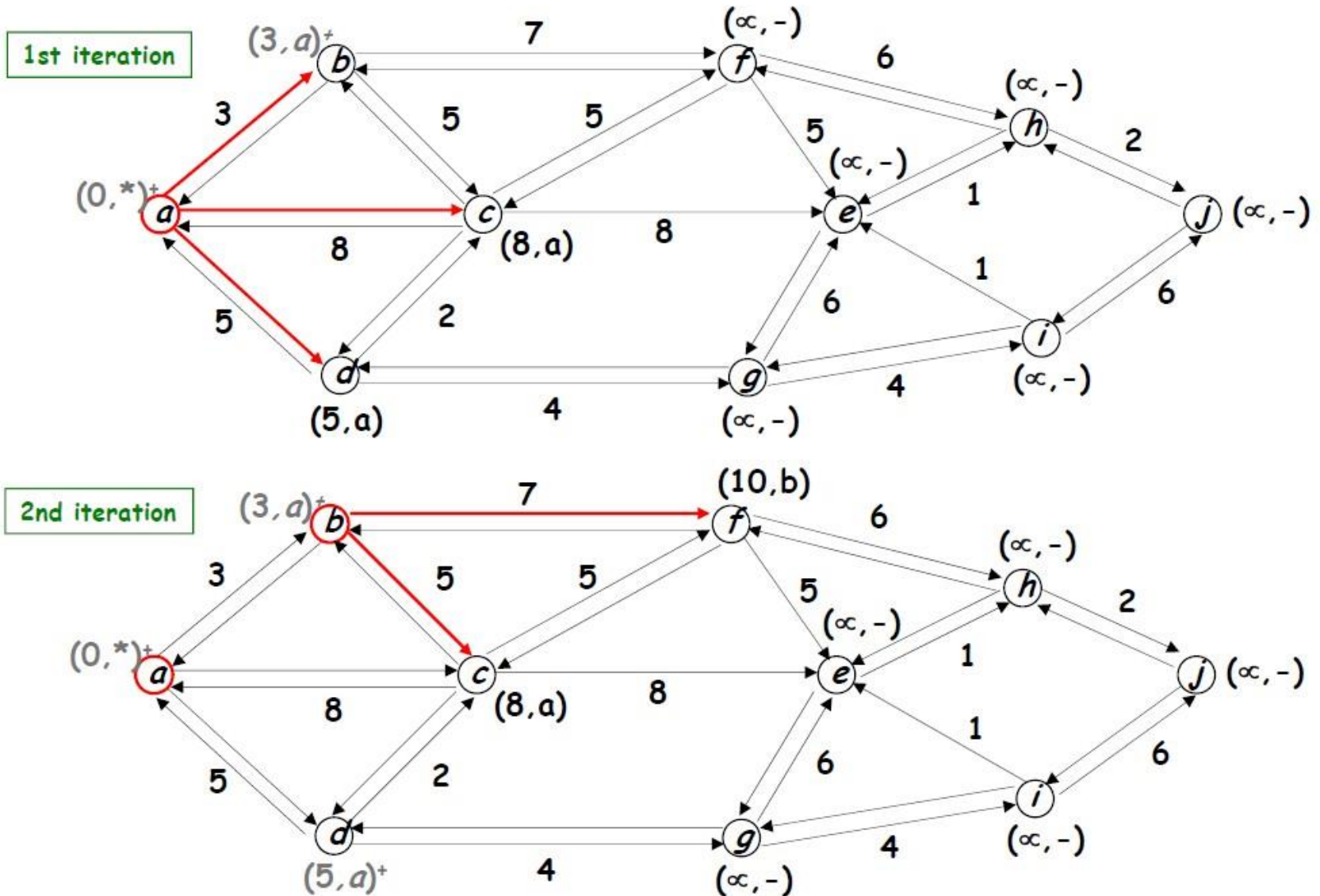


# Dijkstra: Example (Min-cost, Min-Path)



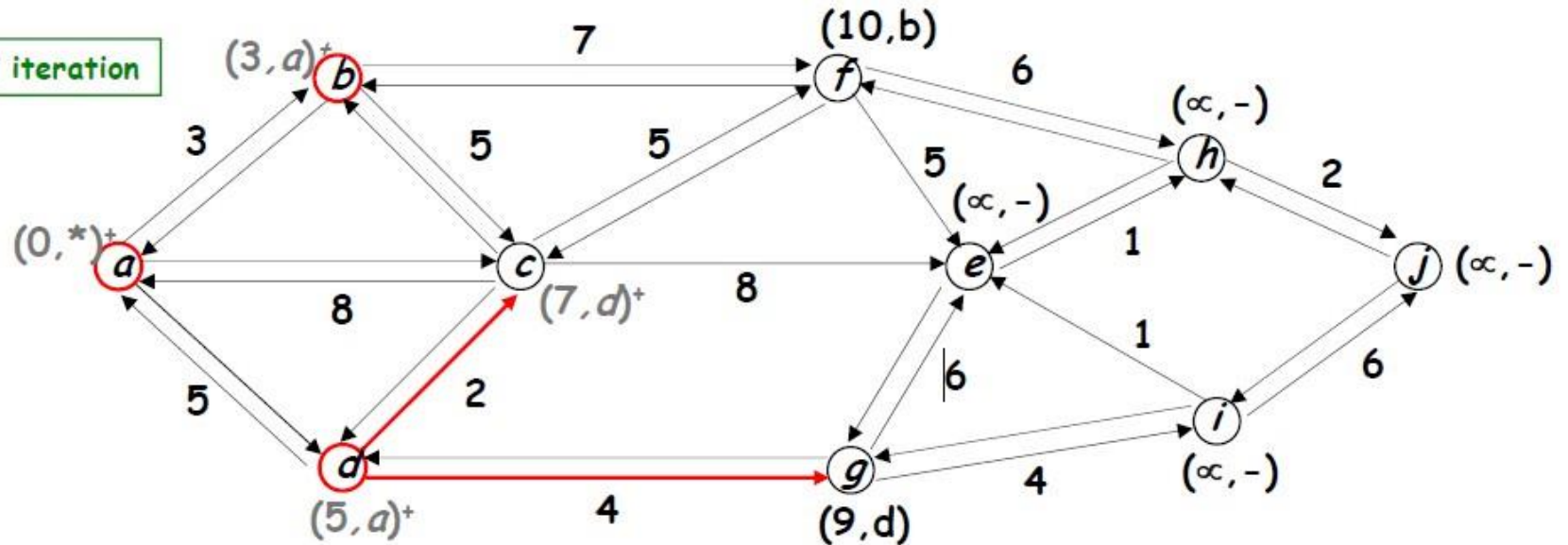
$Q = V, a = \text{DELETE-MIN}(Q)$

# Dijkstra: Example

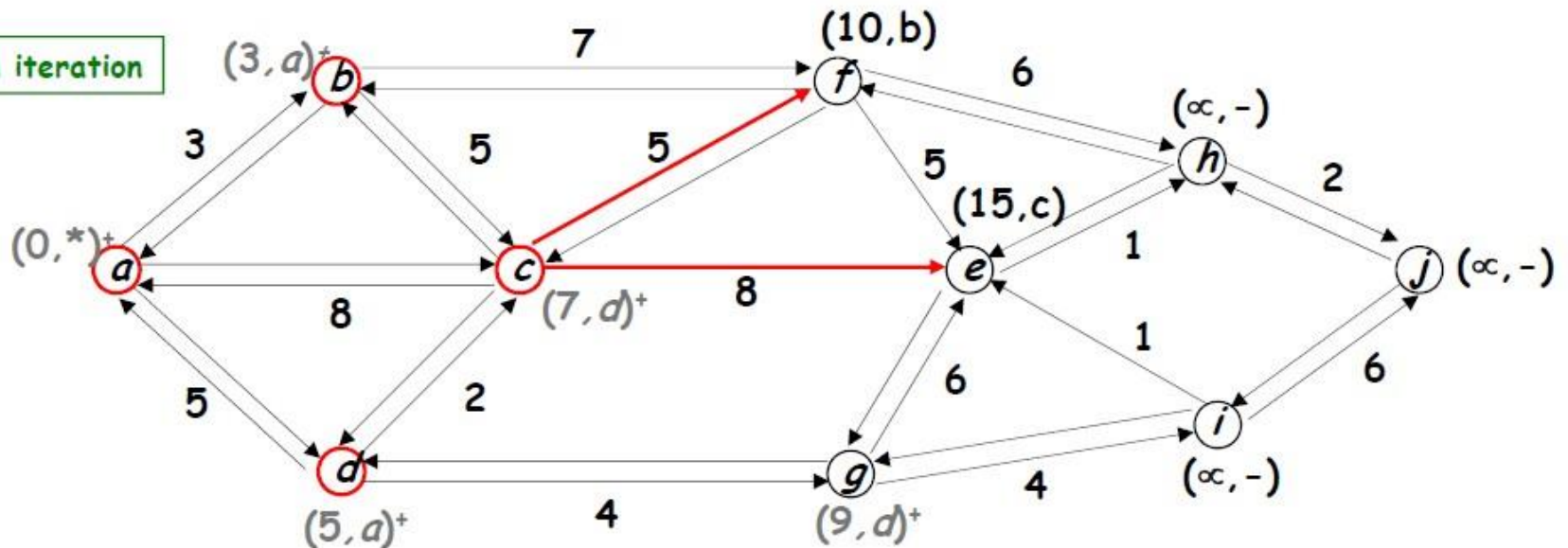


# Dijkstra: Example

3rd iteration

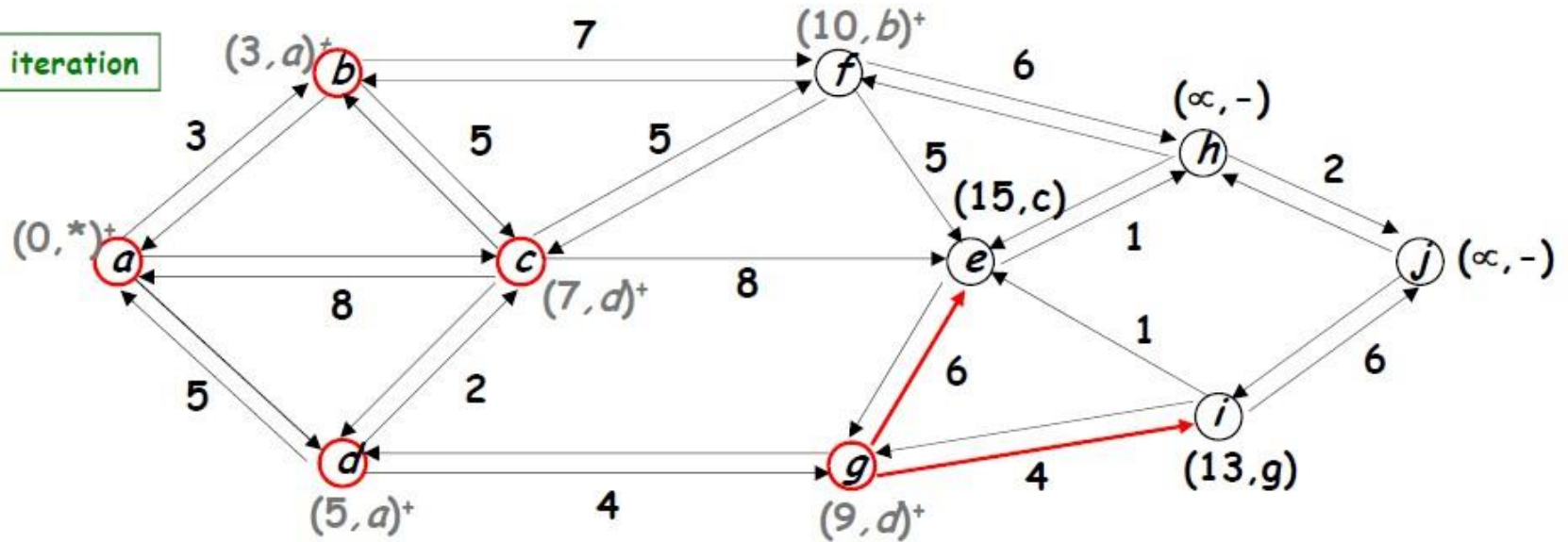


4th iteration

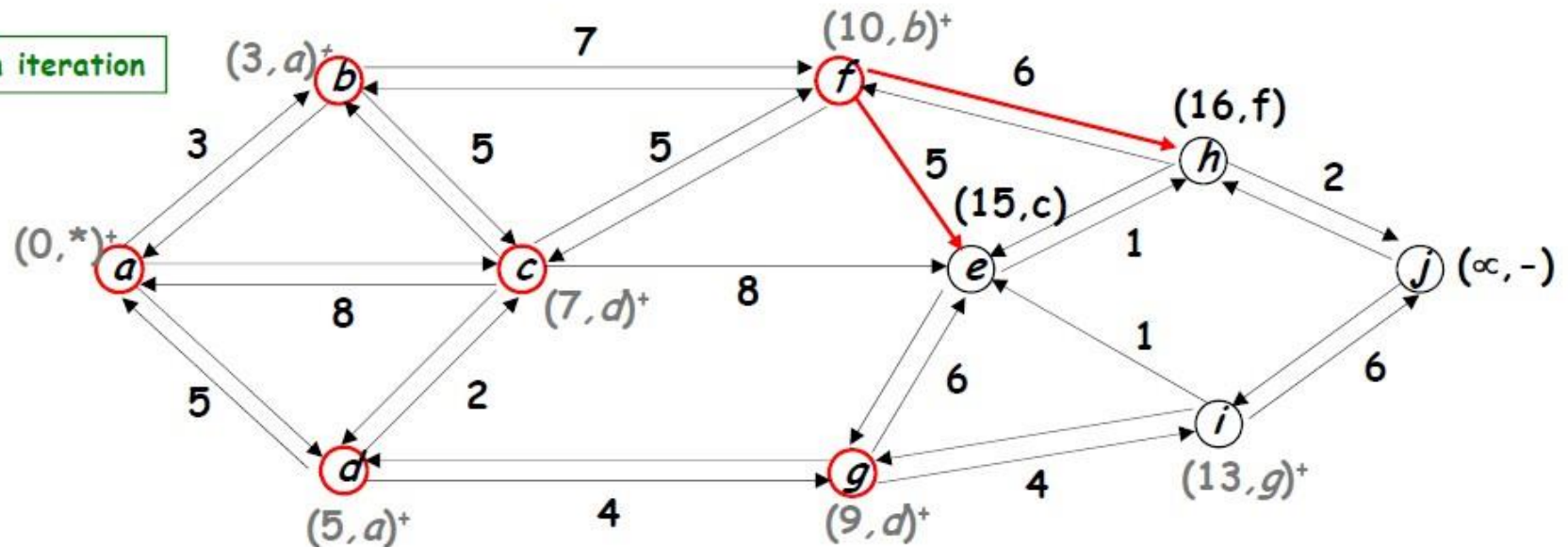


# Dijkstra: Example

5th iteration

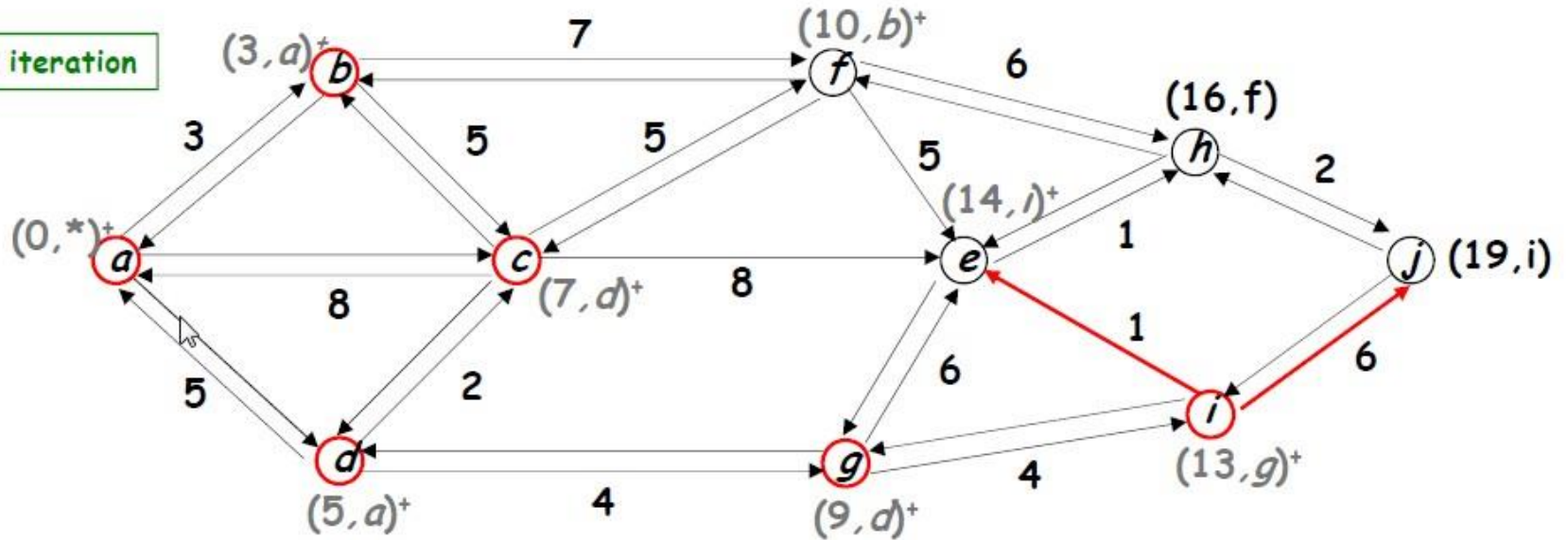


6th iteration

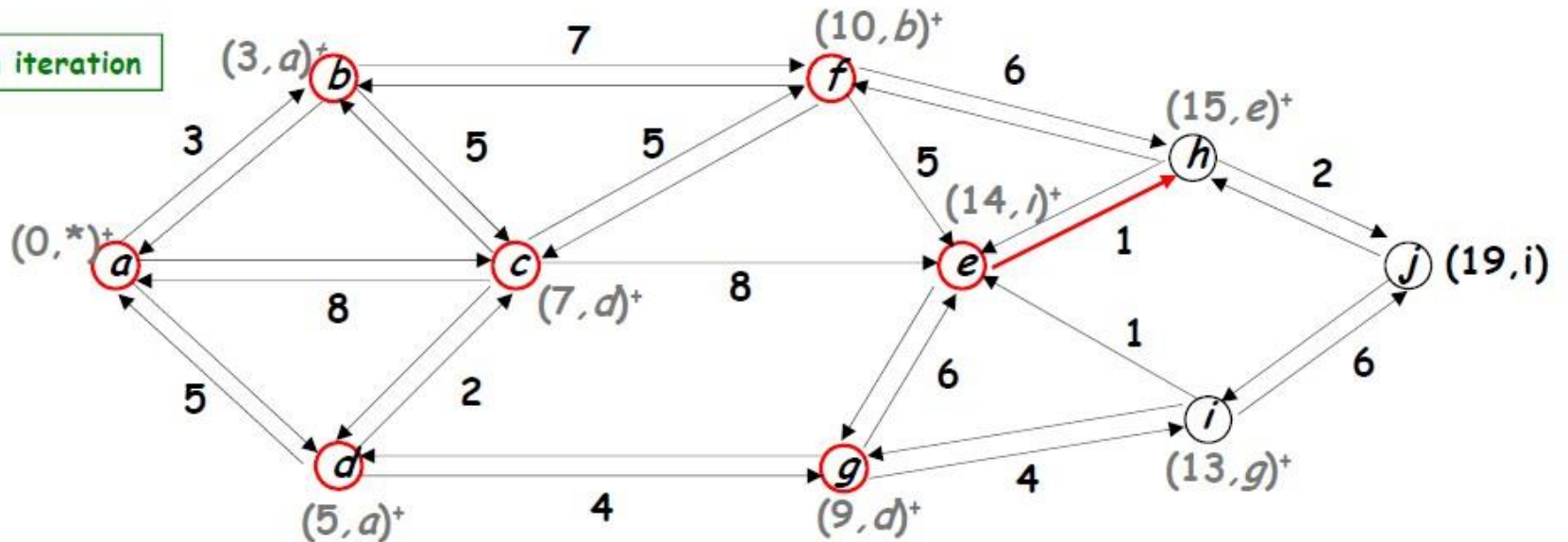


# Dijkstra: Example

7th iteration

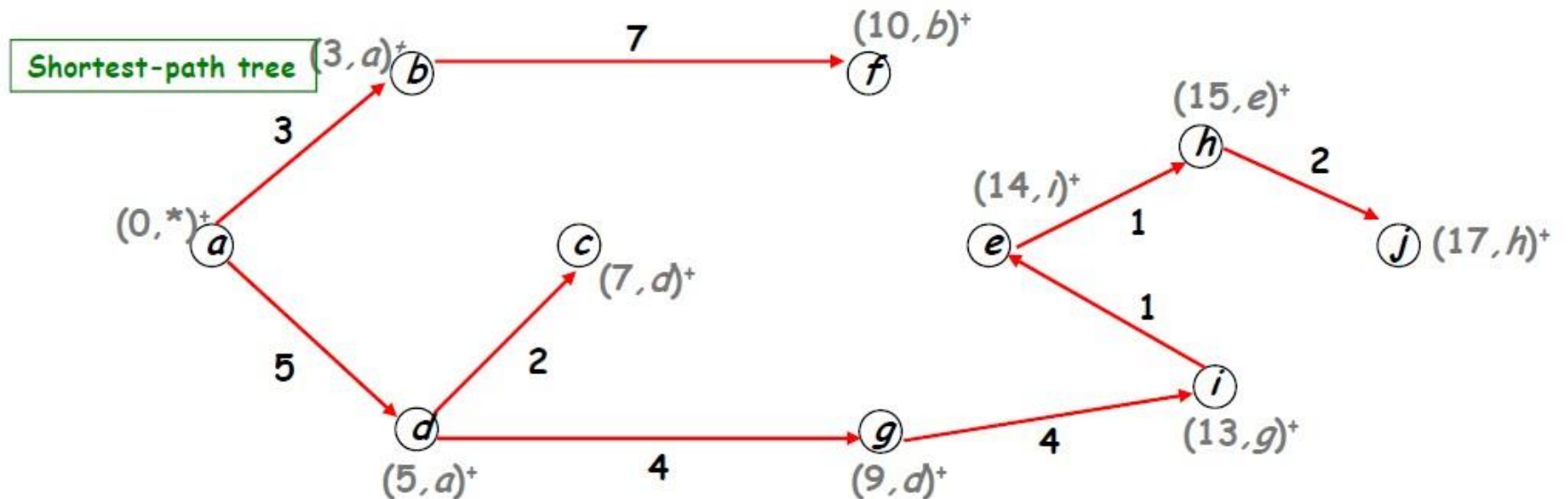
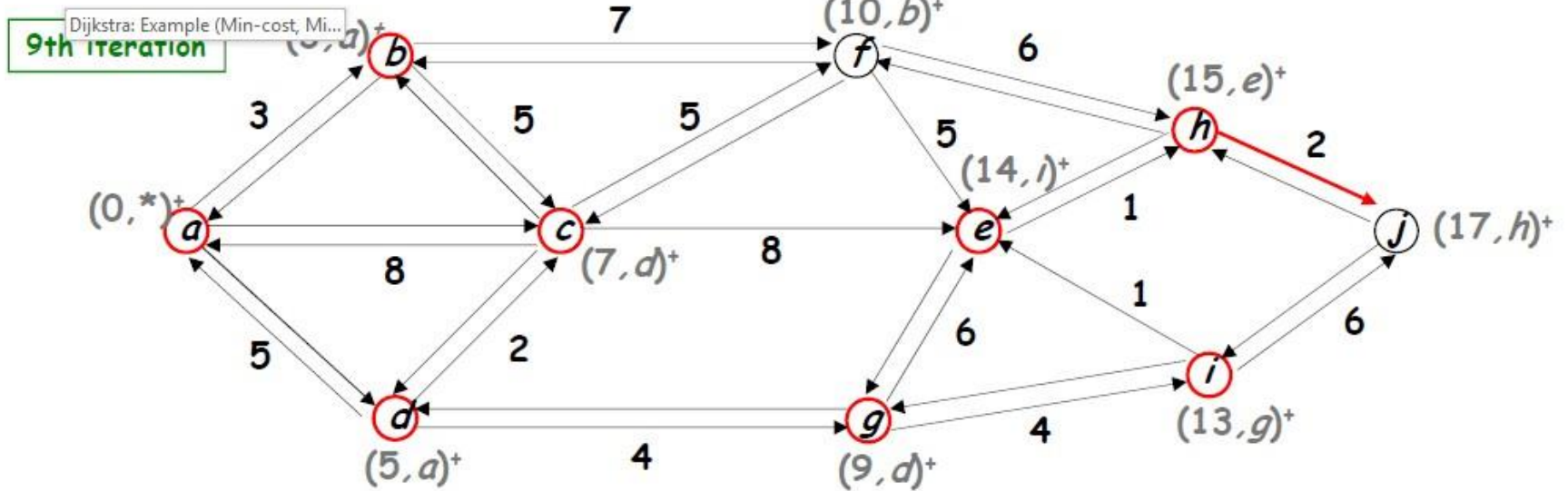


8th iteration





# Dijkstra: Example



# Complexity Analysis of Dijkstra

$|V|$  times { while  $Q \neq \emptyset$   
do  $u \leftarrow \text{DELETE-MIN}(Q)$   
     $S \leftarrow S \cup \{u\}$   
    for each  $v \in \text{Adj}[u]$   
    do if  $d[v] > d[u] + w(u, v)$   
        then  $d[v] \leftarrow d[u] + w(u, v)$

$\text{degree}(u)$  times {

DECREASE-KEY

$$\text{Time} = \Theta(n) \cdot T_{\text{DELETE-MIN}} + \Theta(m) \cdot T_{\text{DECREASE-KEY}}$$





$$\text{Time} = \Theta(n) \cdot T_{\text{DELETE-MIN}} + \Theta(m) \cdot T_{\text{DECREASE-KEY}}$$

| $Q$               | $T_{\text{DELETE-MIN}}$ | $T_{\text{DECREASE-KEY}}$ | Total                          |
|-------------------|-------------------------|---------------------------|--------------------------------|
| array             | $O(n)$                  | $O(1)$                    | $O(n^2)$                       |
| binary<br>heap    | $O(\lg n)$              | $O(\lg n)$                | $O(m \lg n)$                   |
| Fibonacci<br>heap | $O(\lg n)$<br>amortized | $O(1)$<br>amortized       | $O(m + n \lg n)$<br>worst case |

The question of whether computers can think is like the question of whether submarines can swim.

Do only what only you can do.

In their capacity as a tool, computers will be but a ripple on the surface of our culture. In their capacity as intellectual challenge, they are without precedent in the cultural history of mankind.

The use of COBOL cripples the mind; its teaching should, therefore, be regarded as a criminal offence.

APL is a mistake, carried through to perfection. It is the language of the future for the programming techniques of the past: it creates a new generation of coding bums.

Edsger W.  
Dijkstra

